

# OTEL-04: Trace Instrumentation

**Series:** OTEL — OpenTelemetry Integration | **Notebook:** 4 of 8 | **Created:** January 2026 | **Last Updated:** 07/01/2026

## Instrumenting Applications for Distributed Tracing

Traces provide visibility into request flows across services. This notebook covers automatic and manual instrumentation techniques for popular languages with OpenTelemetry.

### Table of Contents

- 1. [Instrumentation Types](#)
- 2. [Auto-Instrumentation](#)
- 3. [Manual Instrumentation](#)
- 4. [Context Propagation](#)
- 5. [Span Attributes and Events](#)
- 6. [Language-Specific Examples](#)
- 7. [Sampling: Head vs. Tail](#)
- 8. [Best Practices](#)

### Prerequisites

| Requirement | Details             |
|-------------|---------------------|
| Knowledge   | OTEL-01 and OTEL-03 |

| Requirement        | Details                               |
|--------------------|---------------------------------------|
| <b>Environment</b> | Collector deployed and accessible     |
| <b>Language</b>    | Examples in Python, Java, Go, Node.js |

## 1. Instrumentation Types

### Comparison

| Type                    | Effort  | Customization | Coverage              |
|-------------------------|---------|---------------|-----------------------|
| <b>Zero-code (auto)</b> | Minimal | Limited       | Framework-level       |
| <b>Manual</b>           | More    | Full          | Custom business logic |
| <b>Hybrid</b>           | Medium  | Balanced      | Best of both          |

### When to Use Each

| Scenario                         | Recommendation                |
|----------------------------------|-------------------------------|
| Quick start, standard frameworks | Auto-instrumentation          |
| Custom business spans            | Manual                        |
| Production with specific needs   | Hybrid                        |
| Serverless / Lambda              | SDK with auto-instrumentation |

### Service Mesh: Istio & Envoy

A service mesh emits telemetry without touching application code — the sidecar proxy (Envoy) generates spans for the traffic it routes. This is zero-code instrumentation at the infrastructure layer, complementary to the SDK patterns above.

- **Envoy** can be configured to emit OpenTelemetry traces directly (an OpenTelemetry tracing provider) and ship them via OTLP — point the exporter at your collector or the Dynatrace OTLP endpoint.
- **Istio** wires this mesh-wide: set the mesh's tracing provider to OpenTelemetry and Istio applies the proxy tracing configuration across the mesh.

Mesh spans cover proxy-to-proxy hops; combine them with app-level SDK spans (via W3C trace-context propagation, §4) for an end-to-end trace. See the Dynatrace Istio/Envoy integration docs for the provider configuration.

Sources: [Istio and Envoy integration \(DT docs\)](#).

## 2. Auto-Instrumentation

---

### Python Auto-Instrumentation

```
# Install
pip install opentelemetry-distro opentelemetry-exporter-otlp
opentelemetry-bootstrap -a install

# Run with auto-instrumentation
opentelemetry-instrument \
  --service_name my-python-app \
  --exporter_otlp_endpoint http://collector:4317 \
  python app.py
```

### Java Auto-Instrumentation

```
# Download agent
curl -L -o opentelemetry-javaagent.jar \
  https://github.com/open-telemetry/opentelemetry-java-
instrumentation/releases/latest/download/opentelemetry-javaagent.jar

# Run with agent
java -javaagent:opentelemetry-javaagent.jar \
  -Dotel.service.name=my-java-app \
  -Dotel.exporter.otlp.endpoint=http://collector:4317 \
  -jar app.jar
```

## Node.js Auto-Instrumentation

```
# Install
npm install @opentelemetry/auto-instrumentations-node \
  @opentelemetry/sdk-node \
  @opentelemetry/exporter-trace-otlp-http

# Run with auto-instrumentation
node --require @opentelemetry/auto-instrumentations-node/register app.js
```

```
// Or programmatically (tracing.js)
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-http');

const sdk = new NodeSDK({
  serviceName: 'my-node-app',
  traceExporter: new OTLPTraceExporter({
    url: 'http://collector:4318/v1/traces',
  }),
  instrumentations: [getNodeAutoInstrumentations()],
});

sdk.start();
```

## 3. Manual Instrumentation

---

### Python Manual Tracing

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.sdk.resources import Resource

# Setup
resource = Resource.create({"service.name": "my-python-app"})
provider = TracerProvider(resource=resource)
processor =
BatchSpanProcessor(OTLPSpanExporter(endpoint="http://collector:4317"))
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)

# Get tracer
tracer = trace.get_tracer(__name__)

# Create spans
@tracer.start_as_current_span("process_order")
def process_order(order_id):
    span = trace.get_current_span()
    span.set_attribute("order.id", order_id)

    with tracer.start_as_current_span("validate_order"):
        # Validation logic
        pass

    with tracer.start_as_current_span("charge_payment"):
        # Payment logic
        pass
```

## Java Manual Tracing

```
import io.opentelemetry.api.GlobalOpenTelemetry;
import io.opentelemetry.api.trace.Span;
import io.opentelemetry.api.trace.Tracer;
import io.opentelemetry.context.Scope;

public class OrderService {
    private static final Tracer tracer =
        GlobalOpenTelemetry.getTracer("order-service");

    public void processOrder(String orderId) {
        Span span = tracer.spanBuilder("process_order").startSpan();
        try (Scope scope = span.makeCurrent()) {
            span.setAttribute("order.id", orderId);

            validateOrder(orderId);
            chargePayment(orderId);
        } finally {
            span.end();
        }
    }
}
```

# Go Manual Tracing

```
import (
    "context"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
)

var tracer = otel.Tracer("order-service")

func ProcessOrder(ctx context.Context, orderID string) error {
    ctx, span := tracer.Start(ctx, "process_order")
    defer span.End()

    span.SetAttributes(attribute.String("order.id", orderID))

    if err := validateOrder(ctx, orderID); err != nil {
        span.RecordError(err)
        return err
    }

    return chargePayment(ctx, orderID)
}
```

## 4. Context Propagation

### W3C Trace Context

The standard propagation format:

| Header      | Example             | Description              |
|-------------|---------------------|--------------------------|
| traceparent | 00-abc123-def456-01 | Trace ID, span ID, flags |
| tracestate  | dt=s:1              | Vendor-specific state    |

### HTTP Propagation

Python (requests):

```

from opentelemetry.propagate import inject
import requests

def call_downstream():
    headers = {}
    inject(headers) # Adds traceparent, tracestate
    response = requests.get("http://downstream/api", headers=headers)

```

### Extracting Context:

```

from opentelemetry.propagate import extract

@app.route('/api/endpoint')
def endpoint():
    context = extract(request.headers)
    with tracer.start_as_current_span("endpoint", context=context):
        # Request handling
        pass

```

## Messaging Propagation

### Kafka:

```

# Producer
from opentelemetry.propagate import inject

headers = []
inject(headers, setter=kafka_header_setter)
producer.send('topic', value=message, headers=headers)

# Consumer
context = extract(record.headers, getter=kafka_header_getter)
with tracer.start_as_current_span("process_message", context=context):
    process(record)

```



## 5. Span Attributes and Events

---

### Adding Attributes

```
with tracer.start_as_current_span("checkout") as span:
    # Business attributes
    span.set_attribute("user.id", user_id)
    span.set_attribute("order.total", order_total)
    span.set_attribute("order.items", len(items))

    # Semantic convention attributes
    span.set_attribute("http.method", "POST")
    span.set_attribute("http.status_code", 200)
```

### Adding Events

```
with tracer.start_as_current_span("payment") as span:
    span.add_event("payment_initiated", {"provider": "stripe"})

    result = process_payment()

    if result.success:
        span.add_event("payment_completed", {
            "transaction_id": result.transaction_id
        })
    else:
        span.add_event("payment_failed", {
            "error": result.error_message
        })
```

### Recording Errors

```
from opentelemetry.trace import StatusCode

with tracer.start_as_current_span("operation") as span:
    try:
        result = risky_operation()
    except Exception as e:
        span.record_exception(e)
        span.set_status(StatusCode.ERROR, str(e))
        raise
```

## 6. Language-Specific Examples

---

### Complete Python Example

```
# app.py
from flask import Flask, request
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.instrumentation.flask import FlaskInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor
from opentelemetry.sdk.resources import Resource

# Setup
resource = Resource.create({"service.name": "checkout-api"})
provider = TracerProvider(resource=resource)
processor = BatchSpanProcessor(
    OTLPSpanExporter(endpoint="http://collector:4318/v1/traces")
)
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)

# Auto-instrument
app = Flask(__name__)
FlaskInstrumentor().instrument_app(app)
RequestsInstrumentor().instrument()

tracer = trace.get_tracer(__name__)

@app.route('/checkout', methods=['POST'])
def checkout():
    with tracer.start_as_current_span("process_checkout") as span:
        order = request.json
        span.set_attribute("order.id", order['id'])

        # Custom business span
        with tracer.start_as_current_span("validate_inventory"):
            validate_inventory(order['items'])

    return {"status": "success"}
```

```
// View spans with custom attributes
fetch spans, from:-1h
| filter isNotNull(order.id)
| fields timestamp, trace.id, span.name, order.id, duration
| sort timestamp desc
| limit 20
```

```
// Find spans with errors
fetch spans, from:-1h
| filter otel.status_code == "ERROR"
| fields timestamp, trace.id, span.name, otel.status_message
| sort timestamp desc
| limit 20
```

## 7. Sampling: Head vs. Tail

Sampling discards a portion of traces to control trace volume and cost. *Where* the decision is made changes what you can decide on.

| Approach             | Where                                | Decision basis                                                 | Trade-off                                                                                           |
|----------------------|--------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Head sampling</b> | SDK, at span start                   | Random / rate-based only — the request outcome isn't known yet | Cheap and simple; keeps/drops blindly, so it can drop the errors and slow traces you most want      |
| <b>Tail sampling</b> | Collector, after the trace completes | The full trace — status, latency, attributes                   | Keeps the interesting traces; costs memory/CPU (the collector buffers each trace until it finishes) |

### Head sampling (SDK)

```
from opentelemetry.sdk.trace.sampling import TraceIdRatioBased

# Keep 10% of traces – decided at the root span, propagated to children
provider = TracerProvider(sampler=TraceIdRatioBased(0.1))
```

Head sampling is consistent across a trace (the root decision propagates), but it cannot prefer errors or slow requests — it does not yet know how the request ends.

## Tail sampling (Collector)

The `tail_sampling` processor buffers each trace until it completes, then applies policies — so you can keep every error and slow trace and probabilistically sample only the healthy remainder:

```
processors:
  tail_sampling:
    decision_wait: 30s          # how long to buffer a trace before deciding
    policies:
      - name: keep-errors
        type: status_code
        status_code: {status_codes: [ERROR, UNSET]}
      - name: keep-slow
        type: latency
        latency: {threshold_ms: 500}
      - name: sample-the-rest
        type: probabilistic
        probabilistic: {sampling_percentage: 20}
```

Tail sampling is **stateful** — all of a trace's spans must reach the **same** collector instance for the decision to see the whole trace. In a tiered deployment (OTEL-03 §5) that means routing by trace ID to the gateway tier; never shard one trace across gateways.

## The trap: sampling biases trace-derived metrics

If OpenTelemetry traces are sampled, trace-derived metrics (request counts, error rates) are computed only from the sampled subset — a 20% sampler makes throughput look 5× lower than reality.

Compute service metrics **before** sampling. The `span_metrics` connector (formerly `spanmetrics` — old name kept as a deprecated alias) reads the full, unsampled trace stream and emits RED metrics; `tail_sampling` then thins only the trace stream that gets stored:

```
connectors:
  span_metrics: {}
  forward/sampling: {}
service:
  pipelines:
    traces/in: # full stream
      receivers: [otlp]
      exporters: [span_metrics, forward/sampling]
    metrics/span_metrics: # metrics from ALL traces (unbiased)
      receivers: [span_metrics]
      exporters: [otlphttp]
    traces/sampled: # only sampled traces are stored
      receivers: [forward/sampling]
      processors: [tail_sampling]
      exporters: [otlphttp]
```

(Representative wiring — see the Dynatrace sampling docs demo for the full pipeline.)

## Dynatrace guidance

- **Do not mix sampling modes on one trace.** Combining OTEL sampling and OneAgent sampling on the same distributed trace yields inconsistent, partial traces — pick one owner per trace (see OTEL-07 §9 signal routing).
- Sampling is a **cost lever, not a default** — start by keeping everything, measure ingest, and sample only if trace volume is the cost driver. OTEL trace volume bills directly under DPS (see OTEL-01 §6 and the FINOPS series).

### Sources:

- [Sampling with the OTEL Collector \(DT docs\)](#) — `tail_sampling` policies  
( `status_code` / `latency` / `probabilistic` , `decision_wait` ), `span_metrics` -before-sampling to avoid metric bias, and the no-mixed-mode warning
- **Derived:** the "route by trace ID to one gateway" requirement combines tail-sampling statefulness with the tiered topology in OTEL-03 §5; the cost-lever framing combines the docs with the DPS trace model (OTEL-01 §6 / FINOPS)

## 8. Best Practices

---

### Span Naming

| Good                              | Bad                              | Why                 |
|-----------------------------------|----------------------------------|---------------------|
| <code>HTTP GET /users/{id}</code> | <code>HTTP GET /users/123</code> | Low cardinality     |
| <code>process_order</code>        | <code>order_abc123</code>        | Descriptive, stable |
| <code>db.query</code>             | <code>SELECT * FROM...</code>    | Operation, not data |

### Attribute Guidelines

| Practice                 | Reason            |
|--------------------------|-------------------|
| Use semantic conventions | Standardization   |
| Keep cardinality low     | Query performance |
| Don't include PII        | Security          |
| Add business context     | Debugging value   |

### Performance Tips

| Tip                   | Impact                |
|-----------------------|-----------------------|
| Use batch processor   | Reduces network calls |
| Sample appropriately  | Reduces volume        |
| Limit attribute size  | Reduces payload       |
| Don't over-instrument | Reduces noise         |

---

## Summary

In this notebook, you learned:

- Instrumentation types: auto, manual, hybrid
  - Auto-instrumentation for Python, Java, Node.js
  - Manual span creation and management
  - Context propagation across services
  - Adding attributes and events to spans
  - Language-specific implementation patterns
  - Best practices for effective tracing
- 

## References

---

- [OTel Python](#)
  - [OTel Java](#)
  - [OTel Go](#)
  - [OTel Node.js](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*